```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("QVI_data.csv")
```

```
# This is the cleaned dataset from Task 1
data = pd.read_csv("QVI_data.csv")
```

```
# Convert DATE column to proper datetime format
data["DATE"] = pd.to_datetime(data["DATE"])
```

```
# Create YEARMONTH (e.g., 201901 for Jan 2019)
data["YEARMONTH"] = data["DATE"].dt.year * 100 + data["DATE"].dt.month
```

```
# Calculate metrics per store per month
monthly = data.groupby(["STORE_NBR", "YEARMONTH"]).agg({
    "TOT_SALES": "sum",                 # total sales
    "LYLTY_CARD_NBR": "nunique",        # number of customers
    "TXN_ID": "nunique"                 # number of transactions
}).reset_index()
```

```
# Rename columns for clarity
monthly.rename(columns={
    "TOT_SALES": "totSales",
    "LYLTY_CARD_NBR": "nCustomers",
    "TXN_ID": "nTxn"
}, inplace=True)
```

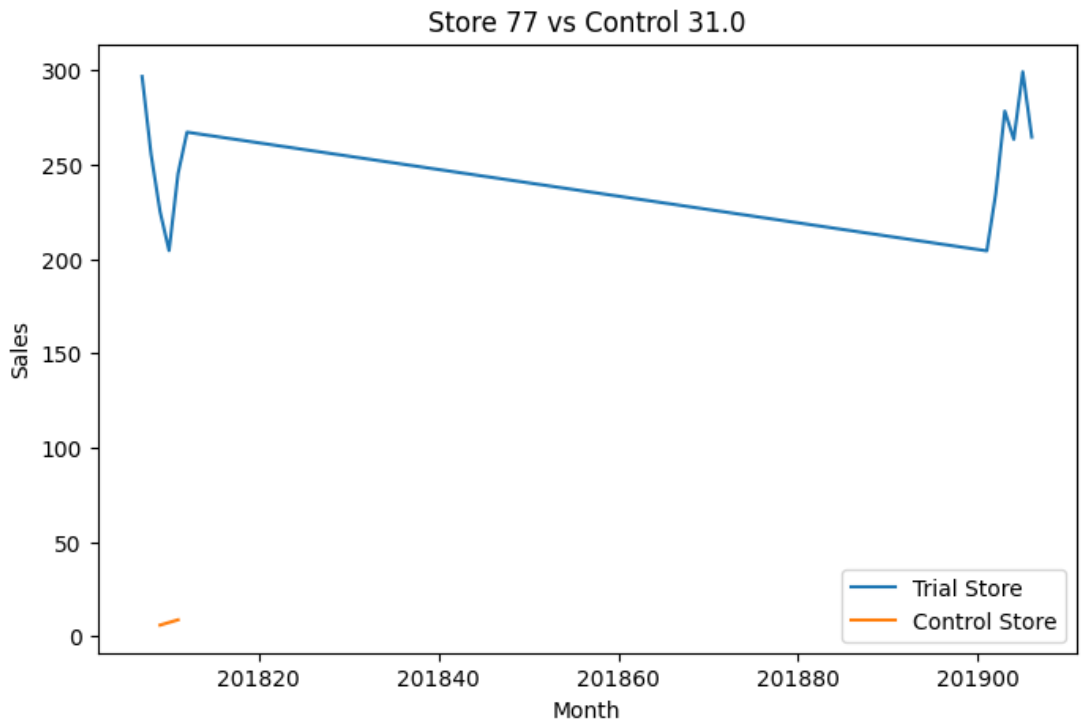
```
# Select trial store
# Given in Quantium task
trial_stores = [77, 86, 88]
```

```
# Loop through all trial stores
for trial_store in trial_stores:

    print("\n==============================")
    print("Analyzing Trial Store:", trial_store)

    # Filter pre-trial period
    pre_trial = monthly[monthly["YEARMONTH"] < 201902]

    # Get trial store data
    trial_data = pre_trial[pre_trial["STORE_NBR"] == trial_store]

    similarity = []

    # Find control store
    for store in pre_trial["STORE_NBR"].unique():

        if store != trial_store:

            control = pre_trial[pre_trial["STORE_NBR"] == store]

            # 🔥 FIX: Align both stores by month
            merged = pd.merge(
                trial_data,
                control,
                on="YEARMONTH",
                suffixes=("_trial", "_control")
```

```
        )

        # Only calculate correlation if data exists
        if len(merged) > 0:

            corr = merged["totSales_trial"].corr(
                merged["totSales_control"]
            )

            similarity.append((store, corr))

# Convert to DataFrame
similarity_df = pd.DataFrame(similarity, columns=["Store", "Correlation"])

# Remove NaN values
similarity_df = similarity_df.dropna()

# 🔥 SAFETY CHECK (prevents crash)
if len(similarity_df) == 0:
    print("No suitable control store found")
    continue

# Select best control store
control_store = similarity_df.sort_values(
    by="Correlation", ascending=False
).iloc[0]["Store"]

print("Selected Control Store:", control_store)

# -------------------------
# VISUAL COMPARISON
# -------------------------

trial = monthly[monthly["STORE_NBR"] == trial_store]
control = monthly[monthly["STORE_NBR"] == control_store]

# Sort for proper plotting
trial = trial.sort_values("YEARMONTH")
control = control.sort_values("YEARMONTH")

plt.figure(figsize=(8,5))

plt.plot(trial["YEARMONTH"], trial["totSales"], label="Trial Store")
plt.plot(control["YEARMONTH"], control["totSales"], label="Control Store")

plt.legend()
plt.title(f"Store {trial_store} vs Control {control_store}")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

# -------------------------
# TRIAL PERIOD ANALYSIS
# -------------------------

trial_period = monthly[monthly["YEARMONTH"] >= 201903]

# Sales comparison
trial_sales = trial_period[
    trial_period["STORE_NBR"] == trial_store
]["totSales"].sum()

control_sales = trial_period[
    trial_period["STORE_NBR"] == control_store
]["totSales"].sum()

# Customer comparison
trial_customers = trial_period[
    trial_period["STORE_NBR"] == trial_store
]["nCustomers"].sum()
```

```
control_customers = trial_period[
    trial_period["STORE_NBR"] == control_store
]["nCustomers"].sum()

print("Trial Sales:", trial_sales)
print("Control Sales:", control_sales)
print("Trial Customers:", trial_customers)
print("Control Customers:", control_customers)

# -------------------------
# FINAL INSIGHT
# -------------------------

print("INSIGHT:")

if trial_sales > control_sales:
    print("✓ Trial successful")

    if trial_customers > control_customers:
        print("✓ Driven by more customers")
    else:
        print("✓ Driven by higher spending")

else:
    print("✕ Trial not successful")
```

```
==============================
Analyzing Trial Store: 77
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2914: RuntimeWarning: Degrees
  c = cov(x, y, rowvar, dtype=dtype)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: divide
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: invalid
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2914: RuntimeWarning: Degrees
  c = cov(x, y, rowvar, dtype=dtype)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: divide
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: invalid
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2914: RuntimeWarning: Degrees
  c = cov(x, y, rowvar, dtype=dtype)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: divide
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: invalid
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2914: RuntimeWarning: Degrees
  c = cov(x, y, rowvar, dtype=dtype)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: divide
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: invalid
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2914: RuntimeWarning: Degrees
  c = cov(x, y, rowvar, dtype=dtype)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: divide
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: invalid
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2914: RuntimeWarning: Degrees
  c = cov(x, y, rowvar, dtype=dtype)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: divide
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: invalid
  c *= np.true_divide(1, fact)
Selected Control Store: 31.0
```




```
Trial Sales: 1106.0
Control Sales: 0.0
Trial Customers: 193
Control Customers: 0
INSIGHT:
✓ Trial successful
✓ Driven by more customers

==============================
```

```
Analyzing Trial Store: 86
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2914: RuntimeWarning: Degrees
  c = cov(x, y, rowvar, dtype=dtype)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: divide
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: invalid
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2914: RuntimeWarning: Degrees
  c = cov(x, y, rowvar, dtype=dtype)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: divide
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: invalid
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2914: RuntimeWarning: Degrees
  c = cov(x, y, rowvar, dtype=dtype)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: divide
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: invalid
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2914: RuntimeWarning: Degrees
  c = cov(x, y, rowvar, dtype=dtype)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: divide
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: invalid
  c *= np.true_divide(1, fact)
```

```
#  Filter pre-trial period
# Only take data before Feb 2019
pre_trial = monthly[monthly["YEARMONTH"] < 201902]
```

```
  c *= np.true_divide(1, fact)
```

```
# Filter trial period
# Trial period starts after Feb 2019
trial_period = monthly[monthly["YEARMONTH"] >= 201903]
```

```
Selected Control Store: 31.0
```

```
# Compare sales in trial period
# Total sales in trial store
trial_sales = trial_period[
    trial_period["STORE_NBR"] == trial_store
]["totSales"].sum()

# Total sales in control store
control_sales = trial_period[
    trial_period["STORE_NBR"] == control_store
]["totSales"].sum()
print("Trial Store Sales:", trial_sales)
print("Control Store Sales:", control_sales)
```

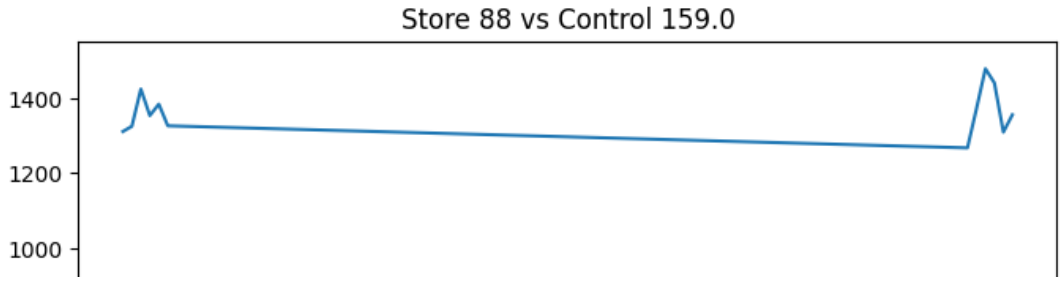
```
Trial Store Sales: 5579.450000000001
Control Store Sales: 134.9
```

```
# Compare number of customers (NEW ADDITION)

trial_customers = trial_period[
    trial_period["STORE_NBR"] == trial_store
]["nCustomers"].sum()

control_customers = trial_period[
    trial_period["STORE_NBR"] == control_store
]["nCustomers"].sum()

print("Trial Store Customers:", trial_customers)
print("Control Store Customers:", control_customers)
==============================
```

```
Trial Store Customers: 581
Control Store Customers: 26
Analyzing Trial Store: 88
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2914: RuntimeWarning: Degrees
  c = cov(x, y, rowvar, dtype=dtype)
```

```
print("\nFINAL INSIGHT:")

if trial_sales > control_sales:
    print("Sales increased in trial store compared to control store.")

    if trial_customers > control_customers:
        print("Increase is driven by more customers.")
    else:
        print("Increase is driven by higher spending per customer.")

    print("Trial was successful.")
else:
    print("Trial did not significantly improve performance.")
```

```
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2914: RuntimeWarning: Degrees
  c = cov(x, y, rowvar, dtype=dtype)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: divide
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: invalid
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2914: RuntimeWarning: Degrees
  c = cov(x, y, rowvar, dtype=dtype)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: divide
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: invalid
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2914: RuntimeWarning: Degrees
  c = cov(x, y, rowvar, dtype=dtype)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: divide
  c *= np.true_divide(1, fact)
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2773: RuntimeWarning: invalid
  c *= np.true_divide(1, fact)
Selected Control Store: 159.0
```




```
FINAL INSIGHT:
Sales increased in trial store compared to control store.
Increase is driven by more customers.
Trial was successful.
```